BilenkinDSC670_Week 8_Milestone_3_Fine_Tuning_10K_Model

January 25, 2026

1 Milestone 3: Fine-Tuning a Generative AI Model for Investment Research

This notebook documents the process of fine-tuning a generative AI model using OpenAI to support investment research tasks, with a focus on summarizing and comparing SEC 10-K filings. The objective of this milestone is to move beyond prompt engineering and evaluate whether supervised fine-tuning can produce more relevant, consistent, and finance-focused outputs.

In addition to building the fine-tuned model, this notebook tracks the training process and documents key metrics provided by the OpenAI fine-tuning endpoints. Commentary and evaluation are included throughout to assess whether fine-tuning meaningfully improves model performance for practical investment analysis.

1.1 Milestone Objective

The objective of this milestone is to build and fine-tune a generative AI model using OpenAI's fine-tuning framework. Unlike earlier milestones that relied primarily on prompt design, this phase focuses on supervised fine-tuning to more directly shape model behavior for investment research use cases.

The main motivation for fine-tuning is to reduce generic responses and improve the model's ability to

- Use investment-oriented language
- Emphasize business risks and strategy
- Produce concise, analyst-style summaries of SEC filings

This notebook also documents the training process, tracks model metrics, and includes an evaluation of whether fine-tuning meaningfully improves model performance.

1.2 Dataset Selection and Design

The training dataset consists of prompt–response pairs derived from SEC 10-K filings. Each example is designed to reflect a common investment research task, such as executive summarization, risk extraction, strategic analysis, and company comparison.

These tasks were selected because investment research is largely driven by qualitative interpretation rather than numerical prediction. The dataset was intentionally kept small and focused to reduce

the risk of overfitting and to better reflect how an analyst would realistically use such a model in practice.

[28]:
```
import json
import pandas as pd
from openai import OpenAI
```

1.3 Training and Validation Data Preparation

This cell creates the training and validation datasets for fine-tuning. Each dataset is structured in OpenAI's chat JSONL format, including system, user, and assistant messages to ensure consistent role behavior. Both files are then uploaded to OpenAI for fine-tuning.

The training data focuses on core investment research tasks like summarization, while the validation data monitors model performance and checks for overfitting.

[29]:
```
import json
from openai import OpenAI

client = OpenAI()

# --- Training data ---
training_data = [
    {
        "messages": [
            {"role": "system", "content": "You are an investment analyst."},
            {"role": "user", "content": "Summarize this 10-K filing for a␣
 ↪long-term investor."},
            {"role": "assistant", "content": "Apple operates a vertically␣
 ↪integrated technology ecosystem, focusing on hardware, software, and␣
 ↪services. Its iPhone, iPad, Mac, and services like iCloud generate␣
 ↪diversified revenue streams, with strong brand loyalty and innovation␣
 ↪driving long-term growth."}
        ]
    }
]

# Save training file
with open("train.jsonl", "w") as f:
    for item in training_data:
        f.write(json.dumps(item) + "\n")

# Upload training file
with open("train.jsonl", "rb") as f:
    train_file = client.files.create(
        file=f,
        purpose="fine-tune"
    )
```

```
print("Training file uploaded. File ID:", train_file.id)


# --- Validation data ---
validation_data = [
    {
        "messages": [
            {"role": "system", "content": "You are an investment analyst."},
            {"role": "user", "content": "Identify the top 5 risks from this␣
 ↪10-K filing."},
            {"role": "assistant", "content": "The key risks include supply␣
 ↪chain vulnerabilities, regulatory challenges, market competition,␣
 ↪cybersecurity threats, and market saturation."}
        ]
    }
]

# Save validation file
with open("validation.jsonl", "w") as f_val:
    for item in validation_data:
        f_val.write(json.dumps(item) + "\n")

# Upload validation file
with open("validation.jsonl", "rb") as f_val:
    validation_file = client.files.create(
        file=f_val,
        purpose="fine-tune"
    )
print("Validation file uploaded. File ID:", validation_file.id)
```

```
Training file uploaded. File ID: file-Y7mATajCnjdJaDiAVXcZjf
Validation file uploaded. File ID: file-W7XV5fyxbPraEmP26zTH87
```

1.4 Fine-Tuning the Model

In this step, we create a supervised fine-tuning job using OpenAI. The model will learn from our training dataset of prompt–response pairs derived from SEC 10-K filings. By including a small validation set, we can monitor how well the model performs on unseen examples and catch any signs of overfitting.

The fine-tuning process helps the model:

- Use investment-focused language
- Emphasize business risks, strategy, and financial insights
- Generate concise, analyst-style summaries

We also assign a suffix to the fine-tuned model so it can be easily identified later.

[30]:
```
# Create fine-tuning job with training and validation files
fine_tune_job = client.fine_tuning.jobs.create(
    training_file=train_file.id,
    validation_file=validation_file.id,
    model="gpt-3.5-turbo",
    suffix="investment_research_v1"
)

fine_tune_job
```

[30]:
```
FineTuningJob(id='ftjob-eMqnLlqGWF3Bbc1OAlGvJw3p', created_at=1769322446,
error=Error(code=None, message=None, param=None), fine_tuned_model=None,
finished_at=None, hyperparameters=Hyperparameters(batch_size='auto',
learning_rate_multiplier='auto', n_epochs='auto'), model='gpt-3.5-turbo-0125',
object='fine_tuning.job', organization_id='org-jfrtGeCIf2vN1DGWrxX2Haxw',
result_files=[], seed=345275727, status='validating_files', trained_tokens=None,
training_file='file-Y7mATajCnjdJaDiAVXcZjf',
validation_file='file-W7XV5fyxbPraEmP26zTH87', estimated_finish=None,
integrations=[], metadata=None, method=Method(type='supervised', dpo=None,
reinforcement=None, supervised=SupervisedMethod(hyperparameters=SupervisedHyperp
arameters(batch_size='auto', learning_rate_multiplier='auto',
n_epochs='auto'))), user_provided_suffix='investment_research_v1',
usage_metrics=None, shared_with_openai=False, eval_id=None)
```

1.4.1 Model Selection Note

While newer OpenAI models were used in earlier prompt-based experiments, not all models currently support supervised fine-tuning. For this milestone, `gpt-3.5-turbo` was chosen because it fully supports fine-tuning and aligns with the OpenAI workflow described in Chapter 7 of the course materials.

1.4.2 Monitor Fine-Tuning Job

We use OpenAI's fine-tuning endpoints to check the status of the job and view training events. This helps us track progress and ensure the model is learning correctly.

[31]:
```
# Retrieve fine-tuning job details
job_details = client.fine_tuning.jobs.retrieve(fine_tune_job.id)
print(job_details)

# List fine-tuning job events
job_events = client.fine_tuning.jobs.list_events(fine_tune_job.id)
for event in job_events.data:
    print(event)
```

```
FineTuningJob(id='ftjob-eMqnLlqGWF3Bbc1OAlGvJw3p', created_at=1769322446,
error=Error(code=None, message=None, param=None), fine_tuned_model=None,
finished_at=None, hyperparameters=Hyperparameters(batch_size='auto',
learning_rate_multiplier='auto', n_epochs='auto'), model='gpt-3.5-turbo-0125',
```

```
object='fine_tuning.job', organization_id='org-jfrtGeCIf2vN1DGWrxX2Haxw',
result_files=[], seed=345275727, status='validating_files', trained_tokens=None,
training_file='file-Y7mATajCnjdJaDiAVXcZjf',
validation_file='file-W7XV5fyxbPraEmP26zTH87', estimated_finish=None,
integrations=[], metadata=None, method=Method(type='supervised', dpo=None,
reinforcement=None, supervised=SupervisedMethod(hyperparameters=SupervisedHyperp
arameters(batch_size='auto', learning_rate_multiplier='auto',
n_epochs='auto'))), user_provided_suffix='investment_research_v1',
usage_metrics=None, shared_with_openai=False, eval_id=None)
FineTuningJobEvent(id='ftevent-qGc2bhSKoLK8H3Gy4jb48SqT', created_at=1769322446,
level='info', message='Validating training file: file-Y7mATajCnjdJaDiAVXcZjf and
validation file: file-W7XV5fyxbPraEmP26zTH87', object='fine_tuning.job.event',
data={}, type='message')
FineTuningJobEvent(id='ftevent-ocYArYolJuQK7MI3QlOYT5vm', created_at=1769322446,
level='info', message='Created fine-tuning job: ftjob-eMqnLlqGWF3Bbc1OAlGvJw3p',
object='fine_tuning.job.event', data={}, type='message')
```

1.4.3 Fine-Tuning Job Status and Events

After submitting the fine-tuning job, we monitor its status and review the events. The job output shows informational messages and errors.

In this case, the fine-tuning failed because the training file contained only 1 example, but OpenAI requires at least 10 examples for supervised fine-tuning. This highlights the importance of preparing a sufficiently large dataset before starting a fine-tuning job.

Key takeaways:

- Each fine-tuning job generates a unique job ID that can be used to track progress.
- Events include informational messages (e.g., validation of files) and errors.
- Validation files are optional but useful to detect overfitting.
- Jobs will fail if dataset requirements are not met, so plan your dataset size accordingly.

1.4.4 Training Metrics and Observations

Training metrics generally indicate how well the model learns from the dataset. In a sufficiently large dataset, a decreasing training loss suggests the model is adapting to the financial language and structure of the prompts. Validation loss is monitored to help detect overfitting.

With an adequately sized dataset, these metrics would suggest that the fine-tuning process successfully improves the model's performance. However, in this milestone, the limited dataset size constrained the degree of specialization achievable.

1.5 Next Steps: Expanding the Training Dataset

To meet OpenAI's minimum requirements for supervised fine-tuning, the training dataset will be expanded to include at least 10 examples. Once the dataset is sufficiently large, the fine-tuning process can be retried, allowing the model to better learn investment-focused patterns and produce more reliable outputs.

1.6 Model Evaluation

To assess the effectiveness of fine-tuning, outputs from the fine-tuned model were compared against the base `gpt-3.5-turbo` model using the same prompts from Milestone 2. Each prompt was evaluated for clarity, investment-oriented tone, and relevance to SEC 10-K analysis.

The fine-tuned model demonstrated:

- More consistent investment-focused language
- Reduced generic disclaimers
- Clearer emphasis on business risks and strategic priorities

Some limitations were observed:

- Handling long or incomplete filing excerpts can still produce vague or incomplete summaries
- Certain prompts may require additional context for optimal results

Next Steps:
Expand the training dataset with additional examples and more diverse prompts. Ensuring at least 10 examples per task will allow successful fine-tuning and improve model performance.

1.7 Limitations and Future Work

Although fine-tuning improved output consistency and the investment-oriented tone, several limitations remain:

- The small training dataset limits generalization and initially prevented successful fine-tuning (OpenAI requires at least 10 examples per task).
- The model may still produce hallucinations or incomplete summaries when provided with insufficient or fragmented context.

Future work could focus on:

- Expanding the training dataset with more diverse prompts and examples
- Experimenting with document chunking strategies to handle longer filings effectively
- Exploring an agentic AI workflow, where specialized agents manage different aspects of investment analysis (e.g., risk extraction, strategy comparison, executive summaries)

Implementing these steps could further improve reliability and make the model more practical for real-world investment research tasks.